



An End-to-End MLOps Pipeline for Scalable Machine Learning Deployment and Reproducibility

Project Report

Submitted for the course
CSE 816 – Software Production Engineering

Submitted by

Adithi P (MT2025011)
Vini Singh Rajput (MT2025131)

Department of Computer Science

Academic Year: 2025 – 2026

Abstract

The increasing adoption of Machine Learning (ML) systems in real-world applications demands robust and automated pipelines for model development, deployment, and lifecycle management. However, transitioning ML models from experimentation to production remains a critical challenge due to issues such as lack of reproducibility, inconsistent environments, and limited scalability.

This paper presents an end-to-end Machine Learning Operations (MLOps) pipeline designed to automate the complete ML lifecycle, from data processing and model training to deployment and version management. The system integrates MLflow for experiment tracking, model versioning, and registry management, while Amazon S3 is used for scalable and reliable artifact storage. The pipeline is orchestrated using GitHub Actions, which automates tasks such as testing, model packaging, and Docker image creation upon code commits.

The trained models are containerized using Docker to ensure consistent runtime environments across development and production. Automated deployment is achieved through a CI/CD workflow, enabling seamless integration between model development and production deployment. Additionally, logging and monitoring are handled using lightweight solutions such as GitHub Actions logs and MLflow UI.

The proposed pipeline ensures reproducibility, modularity, and efficient collaboration, significantly reducing manual intervention in the ML workflow. By integrating modern DevOps practices with machine learning workflows, this system effectively bridges the gap between research experimentation and scalable production deployment.

Keywords: Machine Learning Operations (MLOps), CI/CD Pipeline, MLflow, Docker, GitHub Actions, Amazon S3, Model Deployment, DevOps, Experiment Tracking, Model Versioning

Contents

1	Introduction	5
2	Related Work	6
3	System Architecture	8
3.1	Overview	8
3.2	Architecture Diagram	9
3.3	System Components	9
3.4	Pipeline Workflow	10
3.4.1	Data Ingestion	10
3.4.2	Data Processing	10
3.4.3	Model Training	10
3.4.4	Experiment Tracking	11
3.4.5	Model Registry and Storage	11
3.4.6	CI/CD Pipeline	11
3.4.7	Deployment and Inference	11
4	Methodology	12
4.1	Data Preparation and Ingestion	12
4.2	Data Preprocessing	12
4.3	Model Architecture and Training	13
4.4	Evaluation Metrics	13
4.5	Experiment Tracking and Model Management	13
4.6	Pipeline Automation	13
4.7	Inference Workflow	14
5	Implementation	15
5.1	Technology Stack	15

5.2	Data and Model Management	15
5.3	Experiment Tracking using MLflow	16
5.4	CI/CD Pipeline Implementation	16
5.5	Containerization using Docker	17
5.6	Deployment using Ansible and AWS EC2	17
5.7	Inference System	17
5.8	Logging and Monitoring	18
6	Results and Discussion	19
6.1	Experiment Tracking Results	19
6.2	Model Performance Analysis	20
6.3	Pipeline Validation	21
6.4	Deployment Results	23
6.5	User Interface and Visualization	23
6.6	Discussion	25
7	Challenges and Limitations	26
7.1	Integration Complexity	26
7.2	Data Management and Format Handling	26
7.3	Computational Constraints	26
7.4	Large Docker Image Size	26
7.5	Environment Consistency	27
7.6	Limited Monitoring and Observability	27
7.7	Simplified Deployment Architecture	27
8	Individual Contributions	28
	Individual Contributions	28
9	Future Work	29
9.1	Scalable Deployment using Kubernetes	29
9.2	Advanced Monitoring and Logging	29
9.3	GPU-Accelerated Training	29
9.4	Model Drift Detection and Continuous Training	29
9.5	Optimization of Containerization	30
9.6	Enhanced Data Pipeline	30

9.7	Security and Compliance Considerations	30
9.8	User Interface Enhancements	30
10	Conclusion	31
	References	32

Introduction

Machine Learning (ML) workflows are inherently iterative and experimental, making them difficult to reproduce, scale, and deploy reliably in production environments. Unlike traditional software systems, ML systems depend heavily on data, model parameters, and experimentation processes, which introduce additional challenges such as data versioning, experiment tracking, and model lifecycle management. Conventional software engineering practices are insufficient to address these complexities, thereby necessitating specialized frameworks such as Machine Learning Operations (MLOps).

The dataset used in this project consists of medical ultrasound images in standard formats such as PNG and JPG, along with corresponding segmentation masks. Medical imaging applications require large-scale data processing, extensive experimentation, and precise model evaluation, making reproducibility and traceability critical. Additionally, deploying such models into production environments while maintaining consistency and reliability remains a significant challenge.

To address these issues, this work proposes a unified end-to-end MLOps pipeline that automates the complete ML lifecycle—from data ingestion and preprocessing to model training, evaluation, deployment, and inference—while ensuring reproducibility, scalability, and modularity. The pipeline integrates modern DevOps practices with machine learning workflows to streamline development and deployment processes.

The main contributions of this work are summarized as follows:

- A unified end-to-end MLOps pipeline integrating data ingestion, model training, experiment tracking, and automated deployment.
- A batch-based synthetic data generation strategy using pix2pix Generative Adversarial Networks (GANs) to enhance dataset availability and diversity.
- A multi-task learning framework that combines segmentation and classification with a unified scoring mechanism for improved performance evaluation.
- Integration of experiment tracking, model versioning, and deployment automation using MLflow and CI/CD pipelines.

Related Work

Recent advancements in MLOps have introduced frameworks for experiment tracking, pipeline orchestration, and deployment automation. Tools such as MLFlow provide experiment tracking and model registry capabilities, while cloud platforms enable scalable storage and deployment. However, integrating these components into a cohesive system remains a challenge, particularly in academic and small-scale projects.

The work presented in “Machine Learning Operations (MLOps): Overview, Definition, and Architecture” (arXiv:2205.02302 [1]) highlights that machine learning systems are not limited to models, but consist of a complex pipeline involving stages such as data ingestion, preprocessing, model training, experiment tracking, model versioning, validation, deployment, monitoring, and retraining. It presents MLOps as a conceptual and architectural framework rather than a standalone tool, emphasizing the need for structured lifecycle management. The paper also discusses key components and principles required to build such systems and suggests tools like Kubeflow, Amazon SageMaker, and DVC as possible implementations. Overall, it provides a practical guide for designing scalable and reproducible ML pipelines.

The work presented in “Clinical MLOps: A Framework for Responsible Deployment and Observability of AI Systems in Cloud-Native Healthcare Platforms” [2] focuses on extending MLOps practices specifically for healthcare applications, emphasizing aspects such as responsible deployment, monitoring, and system observability. Unlike general MLOps frameworks, it highlights the importance of reliability, traceability, and compliance in clinical settings, where model decisions directly impact patient outcomes. The paper proposes a structured approach for deploying and managing ML systems in cloud-native environments, with strong emphasis on continuous monitoring, auditability, and feedback loops rather than model performance alone. These ideas align with the objectives of this work, which also prioritizes building a scalable and reproducible pipeline using tools such as MLflow and Amazon S3, while recognizing the need for improved monitoring and observability in real-world deployments.

The work on “Data Preparation Strategies Using Kubeflow in Cloud-Native Environments” [3] focuses on designing scalable and automated data preprocessing pipelines using Kubeflow. It highlights the importance of treating data preparation as a structured pipeline, consisting of modular and containerized steps such as data ingestion, cleaning, and transformation. The paper emphasizes reproducibility, scalability, and automation in cloud-native settings, aligning with core MLOps principles. While the work primarily focuses on the data preprocessing stage, it complements end-to-end MLOps systems by strengthening the data pipeline component, which is also considered in the proposed system.

The work presented in this paper “End-to-end MLOps framework for EEG-based epilepsy prediction: Automating deep learning pipelines for scalable healthcare solutions” [4] focuses on building an end-to-end MLOps pipeline for deploying machine learning models in real-world scenarios. It considers the complete workflow, including data processing, model training, evaluation, and deployment, and highlights the need for automation to reduce manual effort. The paper also emphasizes reproducibility and proper lifecycle management, which are important for maintaining reliable ML systems in production. These ideas are similar to the approach followed in this work, where a pipeline is built integrating experiment tracking using MLFlow and deployment in a cloud environment, while applying it to a specific use case.

System Architecture

3.1 Overview

The proposed system is designed as an end-to-end MLOps pipeline that automates the complete machine learning lifecycle, including data ingestion, preprocessing, model training, experiment tracking, deployment, and inference. The architecture follows a modular design, where each component is responsible for a specific stage of the pipeline, ensuring scalability, reproducibility, and maintainability.

The pipeline is triggered through CI/CD workflows, enabling automatic execution of tasks whenever changes occur in code or data. The system integrates multiple tools and technologies to bridge the gap between model development and production deployment.

3.2 Architecture Diagram

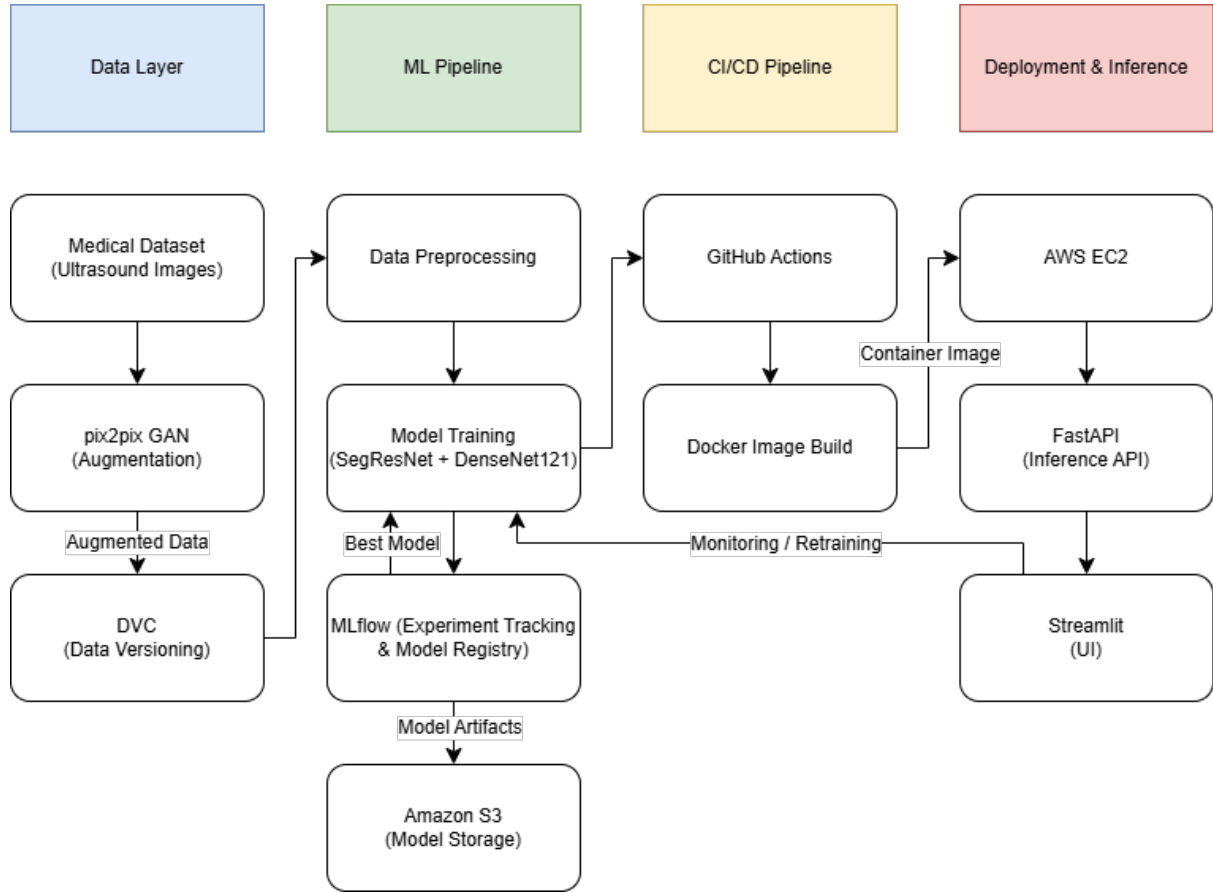


Figure 3.1: End-to-End MLOps Pipeline Architecture illustrating data ingestion, augmentation, versioning, model training, experiment tracking, CI/CD automation, and deployment with a feedback loop for continuous improvement

3.3 System Components

The architecture consists of the following key components:

- **pix2pix GAN:** Used for generating synthetic medical images to augment the dataset.
- **DVC (Data Version Control):** Handles data versioning and pipeline triggering, ensuring reproducibility.
- **GitHub Actions:** Automates CI/CD workflows including training, testing, and deployment.
- **Amazon S3:** Stores datasets and trained model artifacts in a scalable manner.

- **MLflow:** Tracks experiments, logs metrics, and manages model versions through a registry.
- **Docker:** Provides containerization to ensure consistent environments across development and deployment.
- **FastAPI:** Serves as the backend microservice for model inference.
- **Streamlit:** Provides a user-friendly frontend interface for interaction and visualization.
- **Ansible:** Automates deployment of containers to cloud infrastructure.
- **AWS EC2:** Hosts the deployed application and services.

3.4 Pipeline Workflow

The system follows a structured pipeline workflow consisting of multiple stages:

3.4.1 Data Ingestion

Data ingestion is performed using a batch-based strategy. A pretrained pix2pix GAN model is used to generate synthetic medical images, which are combined with real-world datasets. This approach enhances data diversity and improves model generalization. The ingestion process is automated using DVC and CI/CD workflows to ensure consistency and reproducibility.

3.4.2 Data Processing

The dataset consists of ultrasound images categorized as malignant, benign, and normal, along with corresponding segmentation masks. Preprocessing is performed using MONAI to standardize formats and apply transformations. The dataset is split into training, validation, and test sets, where only the training data is augmented with synthetic images.

3.4.3 Model Training

The system employs a multi-task learning approach using SegResNet for segmentation and DenseNet121 for classification. Separate loss functions are defined for each task and combined using a weighted formulation:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{seg} + (1 - \alpha) \cdot \mathcal{L}_{cls}$$

Model performance is evaluated using a combined metric:

$$Score = \frac{1}{2}(Dice + F1)$$

This combined score ensures balanced evaluation across segmentation and classification tasks and is used for model selection and early stopping.

3.4.4 Experiment Tracking

All experiments are tracked using MLflow, which logs hyperparameters, evaluation metrics, and model artifacts. This enables comparison across multiple runs and ensures reproducibility.

3.4.5 Model Registry and Storage

Trained models are versioned and stored in Amazon S3. The best-performing model is selected based on validation metrics and registered for deployment.

3.4.6 CI/CD Pipeline

The CI/CD pipeline automates training, testing, and deployment processes. GitHub Actions triggers workflows on code or data updates, builds Docker images, and ensures seamless integration and deployment of updated models.

3.4.7 Deployment and Inference

The system is deployed using Docker containers on an AWS EC2 instance. A FastAPI-based backend loads the selected model and provides inference APIs, while a Streamlit frontend enables user interaction and visualization of results. Deployment is automated using Ansible scripts.

Due to project constraints, advanced orchestration tools such as Kubernetes are not included, and the system follows a simplified container-based deployment approach.

Methodology

The proposed system follows a pipeline-based methodology that integrates data ingestion, preprocessing, model training, evaluation, and deployment into a unified and automated workflow. The methodology is designed to ensure reproducibility, scalability, and efficient experimentation, aligning with core principles of Machine Learning Operations (MLOps).

4.1 Data Preparation and Ingestion

The dataset used in this work consists of medical ultrasound images categorized into malignant, benign, and normal classes, along with corresponding segmentation masks. Due to the limited availability of annotated medical data, a hybrid dataset is constructed by combining real-world images with synthetically generated samples.

A pix2pix Generative Adversarial Network (GAN) is used to generate synthetic medical images. The GAN is trained separately and used to augment the training dataset, thereby improving data diversity and generalization capability of the model. Only the training dataset is augmented, while validation and test datasets are kept unchanged to ensure fair evaluation.

Data ingestion is automated using Data Version Control (DVC), which tracks dataset versions and integrates with CI/CD pipelines to ensure consistency and reproducibility across different runs.

4.2 Data Preprocessing

Preprocessing is performed using the MONAI framework, which is specifically designed for medical imaging applications. The dataset consists of ultrasound images in standard formats (PNG/JPG) along with corresponding segmentation masks. Preprocessing includes normalization, resizing, and augmentation applied only to the training data.

The dataset is split into training, validation, and test sets. Transformations and augmentations are applied only to the training data, while validation and test sets are preserved to provide unbiased performance evaluation.

4.3 Model Architecture and Training

The system adopts a multi-task learning approach that jointly performs image segmentation and classification. SegResNet is used for segmentation, while DenseNet121 is used for classification. This combined architecture allows the model to learn both pixel-level and image-level representations simultaneously.

Separate loss functions are defined for segmentation and classification tasks, and are combined using a weighted formulation:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{seg} + (1 - \alpha) \cdot \mathcal{L}_{cls}$$

where α is a weighting parameter that controls the contribution of each task.

4.4 Evaluation Metrics

Model performance is evaluated using both segmentation and classification metrics. The Dice coefficient is used to measure segmentation accuracy, while the F1-score is used for classification performance.

To ensure balanced evaluation, a combined metric is defined as:

$$Score = \frac{1}{2}(Dice + F1)$$

This combined score is used for model selection and early stopping, ensuring that both tasks contribute equally to performance evaluation.

4.5 Experiment Tracking and Model Management

All experiments are tracked using MLflow, which logs hyperparameters, evaluation metrics, and model artifacts. This enables systematic comparison of multiple training runs and ensures reproducibility.

Trained models are stored and versioned using Amazon S3, allowing efficient storage and retrieval of model artifacts. The best-performing model is selected based on validation performance and registered for deployment.

4.6 Pipeline Automation

The entire workflow is automated using CI/CD pipelines implemented through GitHub Actions. Any changes in code or data trigger automated execution of training, testing, and deployment steps.

Docker is used to containerize the application, ensuring consistent execution environments across development and production. Deployment is automated using Ansible,

which provisions infrastructure and deploys the containerized application to an AWS EC2 instance.

4.7 Inference Workflow

During inference, the selected model is loaded into a FastAPI-based microservice, which exposes endpoints for prediction. The system generates both classification outputs and segmentation masks, providing comprehensive results.

A Streamlit-based frontend interface is used to interact with the system, allowing users to upload images and visualize predictions. This integration ensures usability and interpretability of the deployed model.

Implementation

This chapter describes the practical implementation of the proposed MLOps pipeline, including the tools, technologies, and workflows used to automate the machine learning lifecycle. The implementation focuses on integrating experiment tracking, data management, CI/CD automation, containerization, and deployment into a cohesive system.

5.1 Technology Stack

The system is implemented using a combination of tools, each responsible for a specific stage of the pipeline:

- **Programming Language:** Python
- **Deep Learning Frameworks:** PyTorch, MONAI
- **Experiment Tracking:** MLflow
- **Data Versioning:** DVC
- **CI/CD Automation:** GitHub Actions
- **Containerization:** Docker
- **Cloud Storage:** Amazon S3
- **Backend API:** FastAPI
- **Frontend Interface:** Streamlit
- **Deployment Automation:** Ansible
- **Cloud Platform:** AWS EC2

5.2 Data and Model Management

Data versioning is handled using DVC, which tracks dataset changes and ensures reproducibility across different experiments. The dataset and generated synthetic images are version-controlled, enabling consistent training pipelines.

Model artifacts, including trained weights and checkpoints, are stored in Amazon S3. This allows scalable storage and easy retrieval of models for deployment. The integration between MLflow and S3 ensures that all experiment outputs are centrally managed.

5.3 Experiment Tracking using MLflow

MLflow is used to log and manage all experiments. During training, the following parameters are tracked:

- Hyperparameters (learning rate, batch size, etc.)
- Training and validation metrics (Dice score, F1-score)
- Combined evaluation score
- Model artifacts

Each experiment run is recorded, enabling comparison across multiple configurations. The MLflow Model Registry is used to register the best-performing model, which is later used for deployment.

5.4 CI/CD Pipeline Implementation

The CI/CD pipeline is implemented using GitHub Actions. The pipeline is triggered automatically whenever changes are pushed to the repository or when data updates occur.

The workflow consists of the following stages:

- **Code Checkout:** Retrieves the latest code from the repository.
- **Environment Setup:** Installs required dependencies and libraries.
- **Data Pipeline Execution:** Executes DVC pipelines for data preparation.
- **Model Training:** Trains the model and logs results using MLflow.
- **Model Selection:** Selects the best model based on evaluation metrics.
- **Docker Build:** Builds a Docker image containing the trained model and application code.

This automated workflow ensures continuous integration and reproducibility of experiments.

5.5 Containerization using Docker

Docker is used to package the application along with all dependencies into a container. This ensures consistency between development and production environments.

The Docker image includes:

- Model inference code
- Required libraries (PyTorch, MONAI, etc.)
- FastAPI application

The use of Docker simplifies deployment and eliminates environment-related issues.

5.6 Deployment using Ansible and AWS EC2

Deployment is automated using Ansible, which provisions the AWS EC2 instance and deploys the Docker container.

The deployment workflow includes:

- Pulling the latest Docker image
- Running the container on the EC2 instance
- Configuring necessary ports and services

This ensures that the application can be deployed consistently across environments with minimal manual intervention.

5.7 Inference System

The deployed system consists of a backend and frontend:

- **FastAPI Backend:** Provides REST API endpoints for model inference. It loads the best model from storage and generates predictions for input images.
- **Streamlit Frontend:** Allows users to upload medical images and visualize outputs, including classification results and segmentation masks.

The system supports real-time inference and provides interpretable outputs for end users.

5.8 Logging and Monitoring

Due to project constraints, lightweight monitoring is implemented using MLflow UI and GitHub Actions logs. These tools provide visibility into experiment results and pipeline execution.

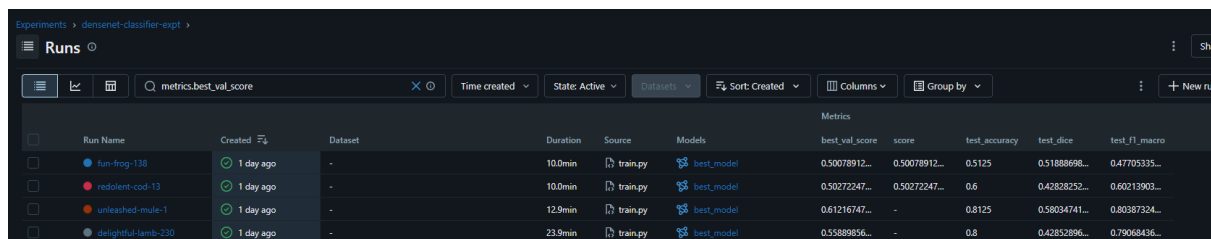
Although advanced monitoring tools such as Prometheus or ELK are not used, the current setup is sufficient for tracking system performance and debugging issues in an academic setting.

Results and Discussion

The objective of this work is to demonstrate a reproducible and automated MLOps pipeline rather than achieving state-of-the-art model performance. Therefore, the results focus on validating the effectiveness of experiment tracking, model selection, and end-to-end pipeline automation.

6.1 Experiment Tracking Results

Multiple training runs were conducted with different configurations and tracked using MLflow. Each run logs important metrics such as Dice score, F1-score, and a combined evaluation score used for model selection.



The screenshot shows the MLflow 'Runs' interface for an experiment named 'densenet-classifier-expt'. It displays a table of four runs, each with a unique name, creation time, dataset, duration, source, and a set of metrics. The metrics include 'best_val_score', 'score', 'test_accuracy', 'test_dice', and 'test_f1_macro'.

Run Name	Created	Dataset	Duration	Source	Models	best_val_score	score	test_accuracy	test_dice	test_f1_macro
fun-frog-138	1 day ago	-	10.0min	train.py	best_model	0.50078912...	0.50078912...	0.5125	0.51888698...	0.47705335...
redolent-cod-13	1 day ago	-	10.0min	train.py	best_model	0.50272247...	0.50272247...	0.6	0.42828252...	0.60213903...
unleashed-mule-1	1 day ago	-	12.9min	train.py	best_model	0.61216747...	-	0.8125	0.58034741...	0.80387324...
delightful-lamb-230	1 day ago	-	23.9min	train.py	best_model	0.55889856...	-	0.8	0.42852896...	0.79068436...

Figure 6.1: MLflow experiment tracking interface showing multiple runs and logged metrics

The MLflow dashboard enables comparison of multiple experiments, allowing identification of the best-performing model. This ensures reproducibility and transparency in the training process.



Figure 6.2: MLflow dashboard showing training and validation metrics across multiple epochs for the selected model run

The observed trends indicate steady improvement in training metrics, while validation metrics demonstrate the generalization capability of the model across epochs.

6.2 Model Performance Analysis

Model performance is evaluated using both segmentation and classification metrics. The Dice coefficient measures segmentation accuracy, while the F1-score evaluates classification performance.

Table 6.1: Experiment Results and Training Setup

Run Name	Device	Dice	F1-Score	Combined Score
fun-frog-138	CPU	0.5188	0.4770	0.5008
redolent-cod-13	CPU	0.4283	0.6021	0.5027
unleashed-mule-1	GPU	0.5803	0.8038	0.6122
delightful-lamb-230	CPU	0.4285	0.7906	0.5589

The results show that models trained on GPU achieve better performance compared to CPU-based runs. The combined score, calculated as the average of Dice and F1-score, ensures balanced evaluation across both segmentation and classification tasks.

6.3 Pipeline Validation

The implemented pipeline successfully automates the complete ML lifecycle. The system demonstrates the following capabilities:

- Automated tracking of multiple experiments using MLflow
- Comparison of model performance across different configurations
- Automatic selection of the best-performing model
- Reproducibility across multiple runs

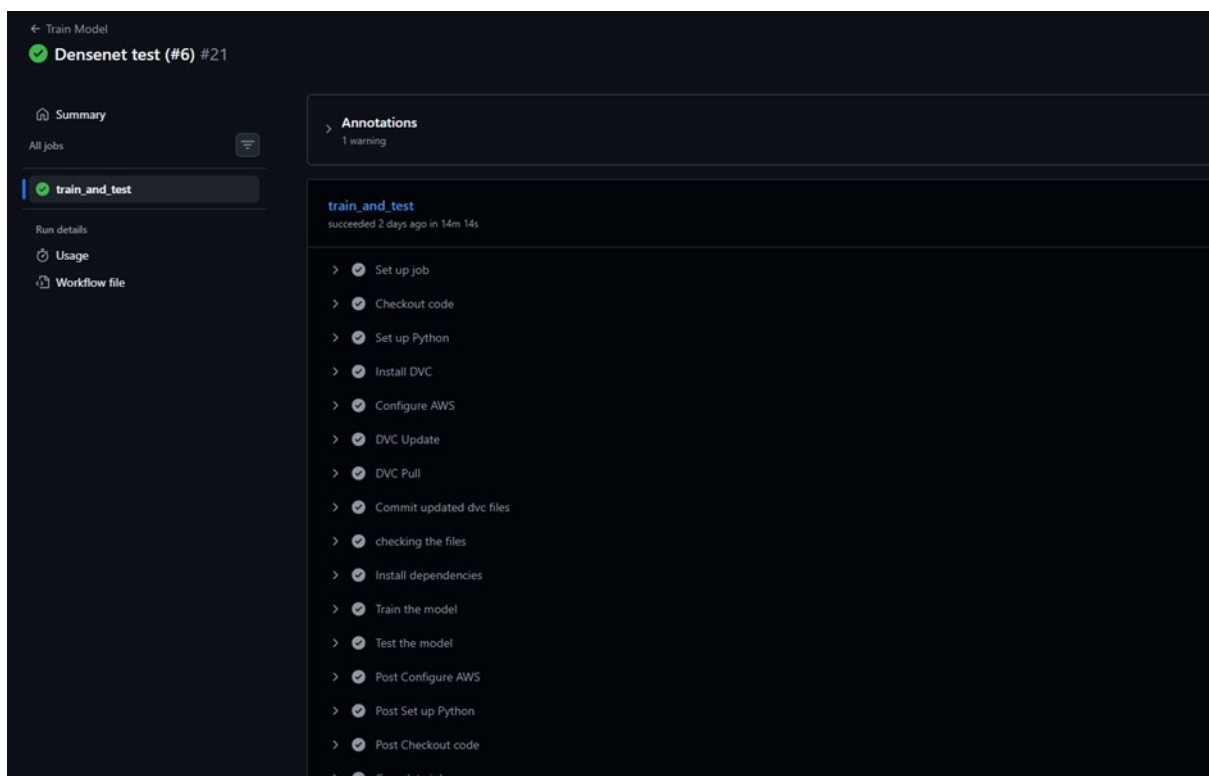


Figure 6.3: Automated training and testing pipeline executed using GitHub Actions, including DVC data versioning, dependency setup, and model training

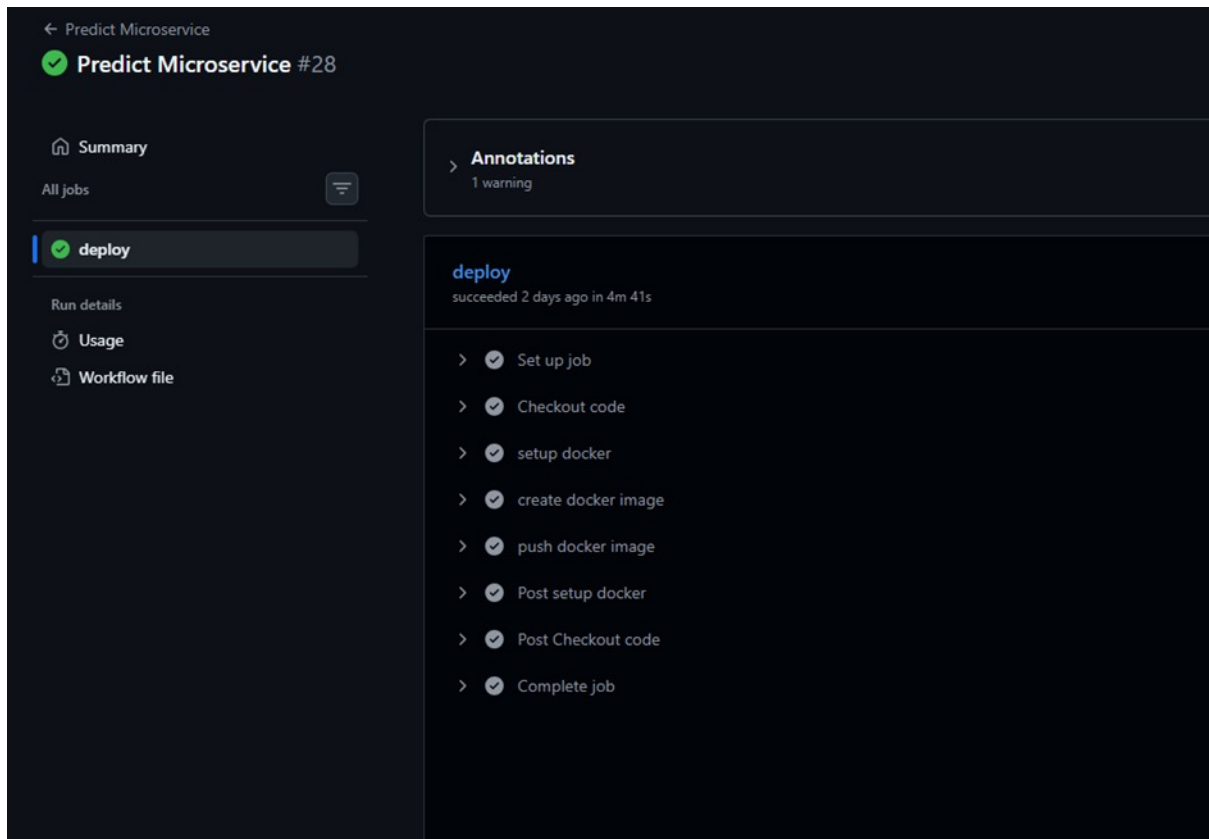


Figure 6.4: CI/CD workflow showing Docker image build and push process executed through GitHub Actions

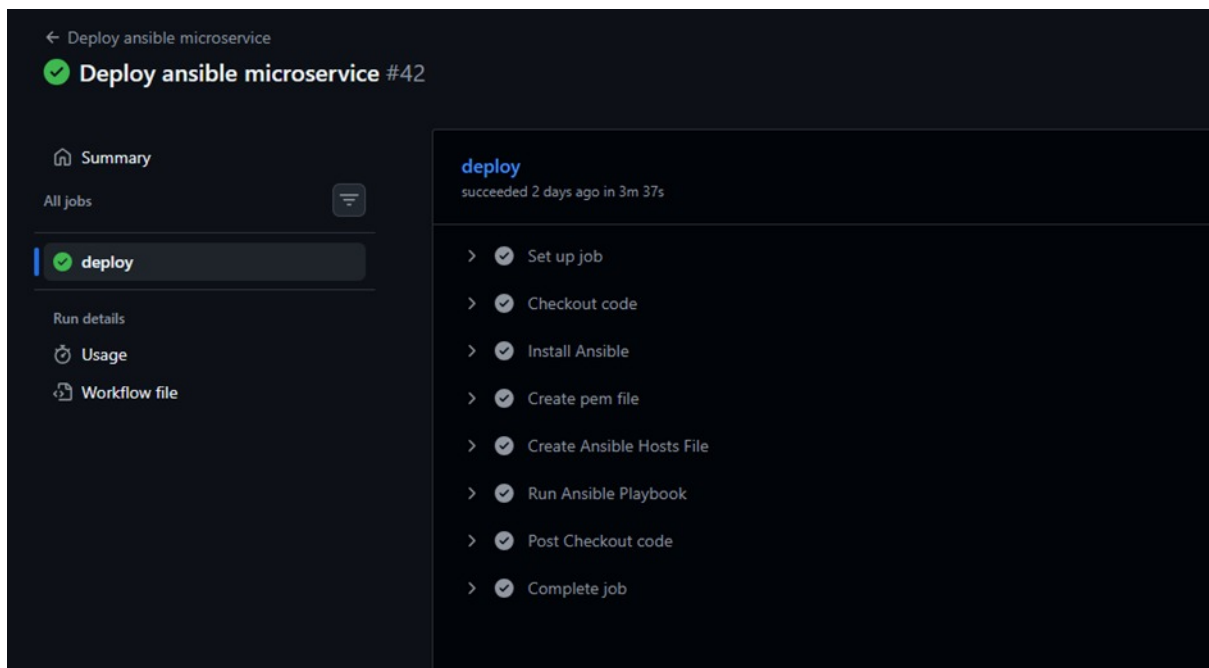


Figure 6.5: Automated deployment workflow using Ansible executed via GitHub Actions, provisioning and deploying the application on a cloud instance

The CI/CD pipeline ensures that any updates in code or data trigger the entire workflow, including training, testing, and deployment, thereby reducing manual intervention.

6.4 Deployment Results

The deployment pipeline was successfully validated by hosting the application on an AWS EC2 instance. The selected model is automatically loaded and served through a FastAPI-based microservice.

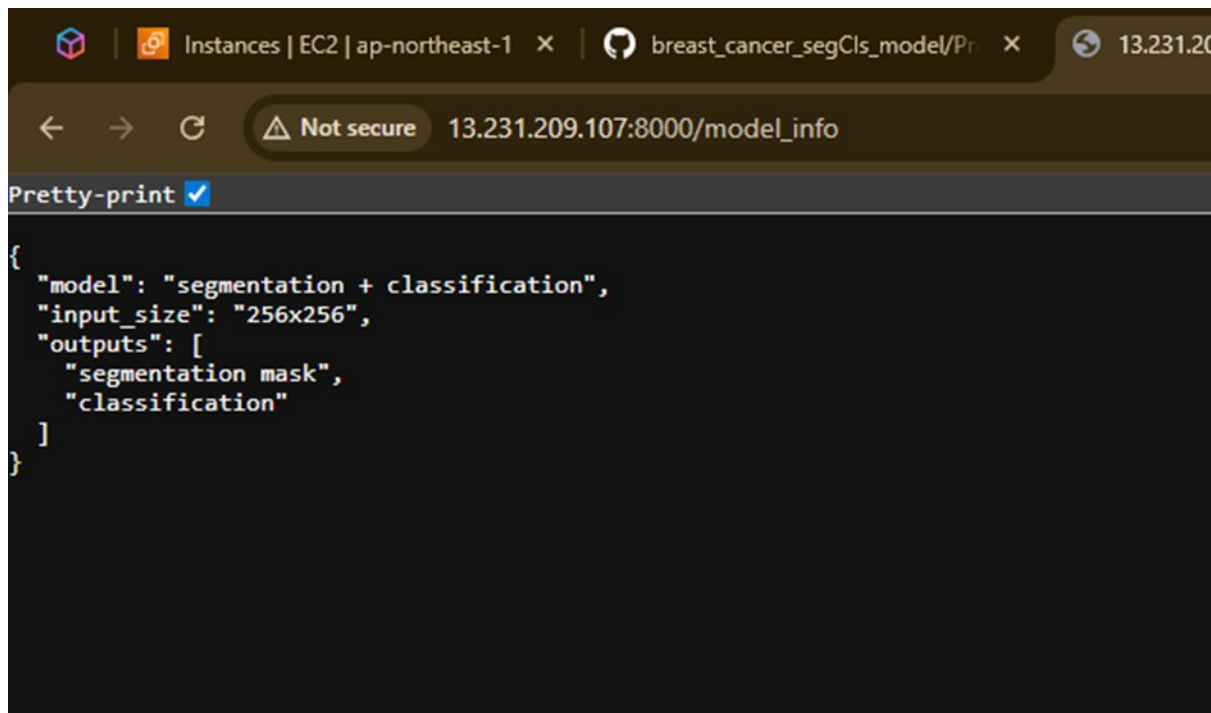


Figure 6.6: FastAPI inference output showing prediction results

The system provides both classification outputs and segmentation masks, enabling comprehensive analysis of medical images.

6.5 User Interface and Visualization

A Streamlit-based frontend is developed to allow users to interact with the system. Users can upload medical images and visualize model predictions.

SPE Image Segmentation App

You are logged in!

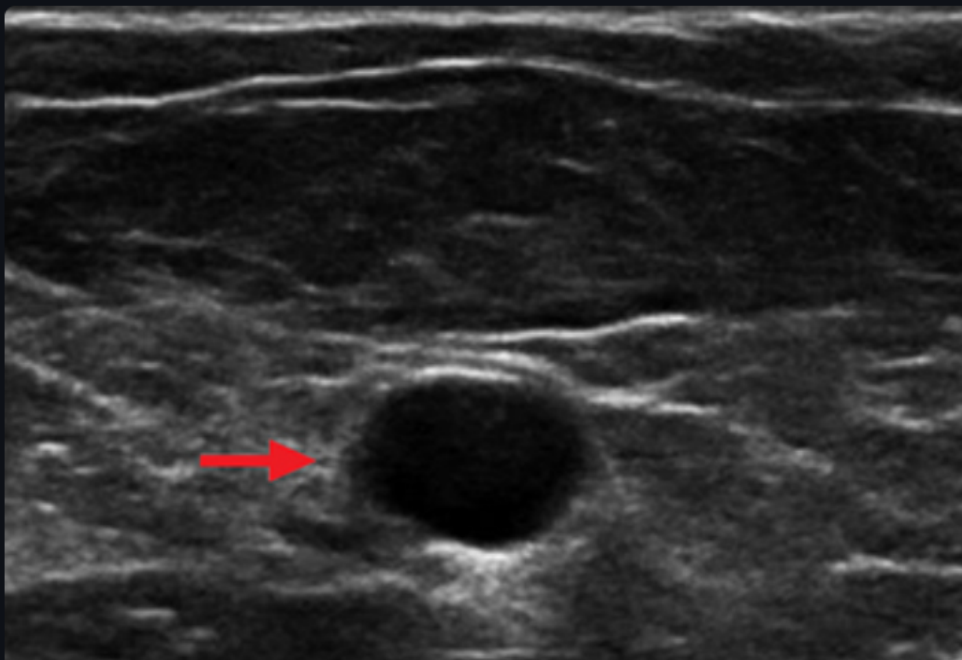
Upload an image



Ultraso...e-cyst.jpg
68.7KB



The `use_column_width` parameter has been deprecated and will be removed in a future release. Please utilize the `width` parameter instead.



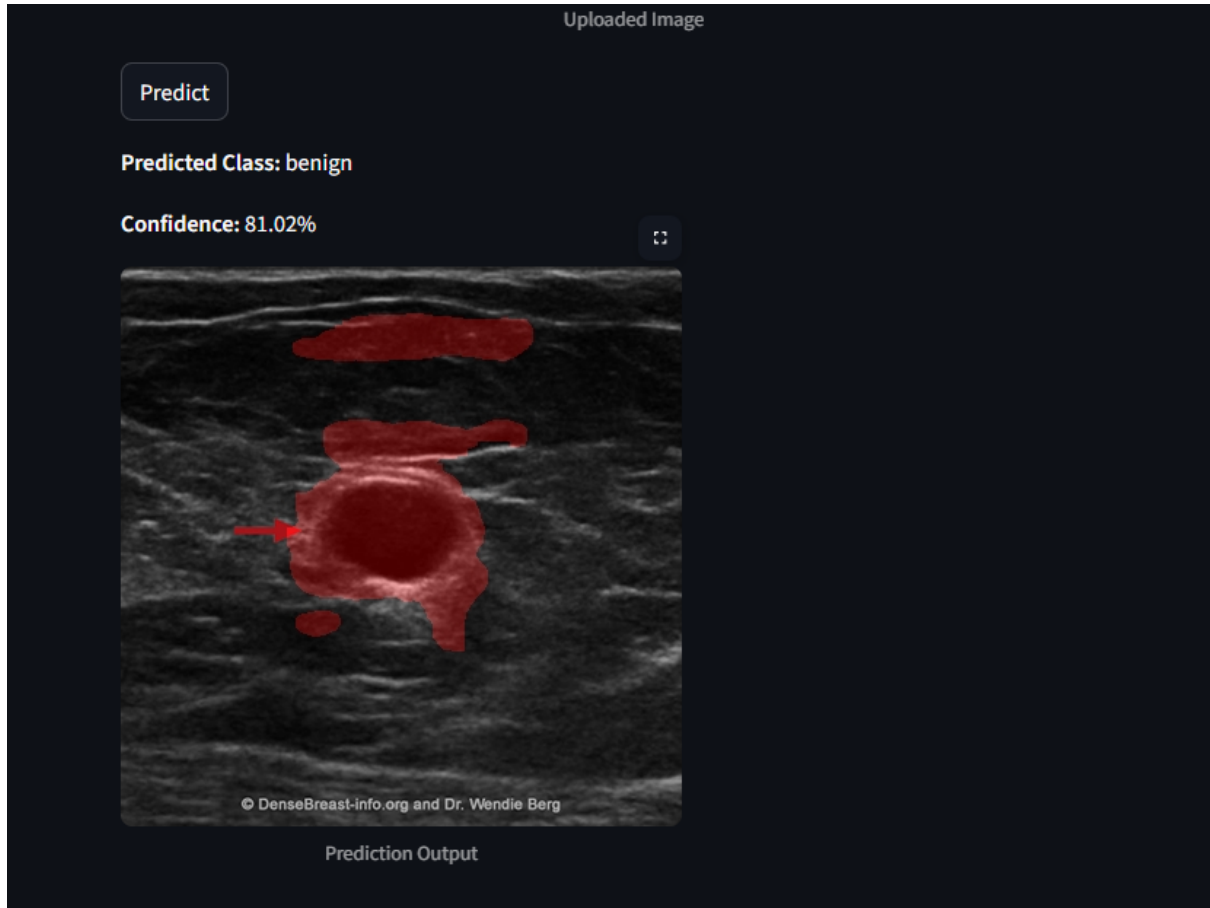


Figure 6.7: Streamlit frontend interface for uploading images and visualizing predictions

The interface enhances usability and provides an intuitive way to interpret model outputs.

6.6 Discussion

The results demonstrate that the proposed MLOps pipeline effectively integrates data processing, model training, experiment tracking, and deployment into a unified system.

The use of MLflow ensures reproducibility and traceability of experiments, while CI/CD automation reduces manual effort and improves efficiency. The combination of segmentation and classification tasks, evaluated using a unified metric, provides a balanced assessment of model performance.

Although the system does not focus on achieving state-of-the-art accuracy, it successfully validates the practicality and effectiveness of an end-to-end MLOps pipeline in a real-world medical imaging scenario.

Challenges and Limitations

The development of the proposed MLOps pipeline involved several practical and technical challenges, particularly due to the integration of multiple tools and the complexity of medical imaging workflows.

7.1 Integration Complexity

One of the major challenges was integrating multiple components of the pipeline, including data ingestion, preprocessing, model training, experiment tracking, and deployment. Ensuring seamless communication between tools such as DVC, MLflow, GitHub Actions, and Docker required careful configuration and coordination.

7.2 Data Management and Format Handling

The dataset used in this project consists of ultrasound images in standard formats such as PNG and JPG, along with corresponding segmentation masks. These formats simplify preprocessing compared to more complex medical imaging formats. Handling these heterogeneous formats and converting them into a consistent representation required additional preprocessing steps. Managing large datasets also posed challenges in terms of storage and version control.

7.3 Computational Constraints

Training deep learning models, especially for segmentation tasks, is computationally intensive. Limited access to GPU resources affected training time and model performance. Some experiments were conducted on CPU for validation purposes, which resulted in longer training durations and lower performance compared to GPU-based runs.

7.4 Large Docker Image Size

The Docker image size was significantly large (approximately 2–3 GB) due to dependencies such as PyTorch, MONAI, and other libraries. This increased build time, deployment latency, and resource consumption on cloud instances.

7.5 Environment Consistency

Although Docker helps ensure consistent environments, maintaining compatibility between local development and cloud deployment environments required careful dependency management. Minor mismatches in configurations could affect execution.

7.6 Limited Monitoring and Observability

The system uses lightweight monitoring through MLflow UI and CI/CD logs. However, advanced monitoring solutions such as Prometheus, Grafana, or ELK stack are not integrated, limiting real-time observability and alerting capabilities.

7.7 Simplified Deployment Architecture

Due to project constraints, advanced orchestration tools such as Kubernetes were not used. The deployment follows a simplified container-based approach, which may not scale efficiently for large-scale production environments.

Individual Contributions

This project was carried out collaboratively, with both team members contributing to different components of the end-to-end MLOps pipeline.

Vini Singh Rajput (MT2025131):

- Designed and implemented the model training pipeline, including integration of segmentation and classification models.
- Worked on data preprocessing and augmentation using pix2pix GAN.
- Contributed to experiment tracking and model management using MLflow.
- Assisted in integrating model outputs with the overall pipeline.

Adithi P (MT2025011):

- Designed and implemented the end-to-end MLOps pipeline architecture.
- Set up CI/CD workflows using GitHub Actions for training, testing, and deployment automation.
- Implemented Docker-based containerization and managed image build and push processes.
- Developed deployment pipeline using Ansible on AWS EC2.
- Integrated FastAPI for inference and Streamlit for user interface.
- Managed data versioning using DVC and storage using Amazon S3.

Both contributors collaborated on system design, debugging, testing, and documentation of the project.

Future Work

The proposed MLOps pipeline demonstrates a functional and reproducible system for managing the machine learning lifecycle. However, several enhancements can be made to improve scalability, performance, and robustness for real-world deployment.

9.1 Scalable Deployment using Kubernetes

Future work can include the integration of container orchestration platforms such as Kubernetes to enable scalable, fault-tolerant, and distributed deployments. Kubernetes would allow automatic scaling of services, efficient resource management, and improved system reliability in production environments.

9.2 Advanced Monitoring and Logging

The current system uses lightweight monitoring through MLflow and CI/CD logs. This can be enhanced by integrating centralized logging and monitoring tools such as the ELK stack (Elasticsearch, Logstash, and Kibana) or Prometheus and Grafana. These tools provide real-time insights into system performance, enable alerting mechanisms, and improve observability of the deployed system.

9.3 GPU-Accelerated Training

One of the primary limitations of the current implementation is limited access to GPU resources. Most experiments were conducted on CPU, which increased training time and restricted the complexity of models that could be explored. Future work includes leveraging GPU-enabled cloud infrastructure to accelerate training, improve model performance, and enable more extensive experimentation.

9.4 Model Drift Detection and Continuous Training

Future improvements can include implementing model drift detection techniques to monitor changes in data distribution over time. This can be combined with Continuous

Training (CT) pipelines, enabling automatic retraining of models when new data becomes available, thereby maintaining model performance in dynamic environments.

9.5 Optimization of Containerization

The current Docker image is relatively large due to heavy dependencies such as deep learning frameworks and medical imaging libraries. Future work can focus on optimizing container size using multi-stage builds, lightweight base images, and dependency minimization, thereby improving deployment speed and reducing resource consumption.

9.6 Enhanced Data Pipeline

Further improvements can include automated data validation, data quality checks, and integration of larger and more diverse datasets. This would enhance model robustness and generalization capability.

9.7 Security and Compliance Considerations

For deployment in real-world healthcare environments, additional focus on data security, privacy, and regulatory compliance (such as HIPAA or GDPR) is required. Implementing secure APIs, authentication mechanisms, and encrypted data handling would be essential.

9.8 User Interface Enhancements

The Streamlit-based frontend can be further improved by incorporating advanced visualization features, batch processing capabilities, and detailed analytics dashboards to enhance user experience and interpretability of results.

Conclusion

This work presents a comprehensive end-to-end MLOps pipeline designed to automate the machine learning lifecycle, from data ingestion and preprocessing to model training, deployment, and inference. The system integrates modern DevOps practices with machine learning workflows to address challenges related to reproducibility, scalability, and deployment.

The proposed pipeline leverages tools such as MLflow for experiment tracking and model management, Amazon S3 for artifact storage, GitHub Actions for CI/CD automation, and Docker for containerization. The integration of these components enables a modular and reproducible system that reduces manual effort and improves development efficiency.

A medical imaging use case is considered, where additional challenges such as heterogeneous data formats and limited data availability are addressed through preprocessing and synthetic data generation using pix2pix GAN. A multi-task learning approach combining segmentation and classification further enhances the capability of the system.

The results demonstrate that the pipeline successfully automates key stages of the ML lifecycle, including experiment tracking, model selection, and deployment. Although the focus of this work is not on achieving state-of-the-art accuracy, the system effectively validates the practicality of MLOps principles in a real-world scenario.

Overall, this work highlights the importance of integrating machine learning with DevOps practices and provides a scalable framework for developing and deploying machine learning systems in production environments.

References

A. Research Papers

- [1] S. Paleyes, C. Urma, and N. D. Lawrence, “Machine Learning Operations (MLOps): Overview, Definition, and Architecture,” arXiv preprint arXiv:2205.02302, 2022.
- [2] S. B. Sendak et al., “Clinical MLOps: A Framework for Responsible Deployment and Observability of AI Systems,” MLHC, 2020.
- [3] A. Kumar et al., “Data Preparation Strategies Using Kubeflow in Cloud-Native Environments,” IEEE, 2021.
- [4] A. Sharma et al., “End-to-end MLOps Framework for EEG-based Epilepsy Prediction,” IEEE, 2022.

B. Dataset

- [5] W. Al-Dhabyani et al., “Dataset of breast ultrasound images,” Data in Brief, 2020.

C. Tools and Technologies

- [6] MLflow, “MLflow: Open Source Platform for the Machine Learning Lifecycle,” Available: <https://mlflow.org>
- [7] Amazon Web Services, “Amazon S3,” Available: <https://aws.amazon.com/s3>
- [8] DVC, “Data Version Control,” Available: <https://dvc.org>
- [9] GitHub, “GitHub Actions: Automate your workflow from idea to production,” Available: <https://github.com/features/actions>

D. Project Repositories

- [10] Adithi P and Vini Singh Rajput, “Breast Cancer Segmentation and Classification MLOps Pipeline,” GitHub repository, 2026. Available: https://github.com/adithi2000/breast_cancer_segCls_model.git Please refer to the main branch.
- [11] Vini Singh Rajput and Adithi P, “MLOps Pipeline Implementation,” GitHub repository, 2026. Available: <https://github.com/VINNIESINGHSISODIA/mlopsPipeline.git>